



UNIVERSIDAD NACIONAL DE LOJA

CARRERA DE COMPUTACIÓN

Grupo:

8

Integrantes:

Jean Carlos Aguirre Romero

Irvin Alexey Armijos Guerra

Galo Milton Benítez Cabrera

Héctor Leonardo Guerrero Espinoza

Axel Joao Jiménez Palta

Erick Jahirs Rogel Baque

Asignatura:

Programación Orientada a Objetos

Docente:

Wilman Chamba Zaragocín

LOJA – LOJA - ECUADOR

1. Programación Limpia (Clean Code) en Java

Lo que se conoce como Clean Code o código limpio se refiere a escribir el código de manera sencilla de leer, entender y mantener por cualquier persona. Se basa en la premisa de que el código se lee muchas más veces de las que se escribe [1].

- **Principios Clave del Clean Code**

- A. **SOLID:** Son 5 principios del diseñado orientado a objetos que buscan hacer el software más mantenible y escalable [1].

- I. **Responsabilidad Única:** Una clase debe tener solo una razón para cambiar.

- II. **Abierto/Cerrado:** El software debe estar abierto para extensión, pero cerrado para modificación.

- III. **Sustitución de Liskov:** Las subclases deben ser sustituibles por sus clases base sin alterar el funcionamiento.

- IV. **Segregación de Interfaces:** Es mejor tener muchas interfaces específicas que una sola interfaz de propósito general.

- V. **Inversión de Dependencias:** Se debe depender de abstracciones, no de concreciones.

- B. **DRY (Don't Repeat Yourself):** Este principio establece que se debe evitar la duplicación de código. Si se hace lo mismo en dos lugares, se debe extraer a un solo método o clase, reduciendo la posibilidad de inconsistencias [2].

- C. **KISS (Keep It Simple, Stupid):** Los sistemas funcionan mejor si se mantienen simples en lugar de hacerse complejos. Se debe evitar la sobreingeniería y buscar la solución más directa al problema [1] [2].

- **Beneficios**

Aplicar Clean Code trae consigo muchas ventajas, de las cuales se pueden destacar: menos tiempo invertido en descifrar código complejo o en arreglar *bugs* introducidos por mala estructura. Permite incorporar nuevas funcionalidades de forma más segura.

Los nuevos desarrolladores pueden entender el flujo de la aplicación más rápido sin necesidad de explicaciones extensas. Al tener funciones pequeñas y clases cohesionadas, es mucho más fácil implementar pruebas unitarias (Unit Testing) efectivas [1] [2].

- **Técnicas y convenciones de Clean Code en Java:**

- 1) **Nomenclatura para:**

- ❖ **Clases:** Deben ser sustantivos en formato *PascalCase* y no deben ser verbos. (ej. GestorUsuarios, CuentaBancaria).

- ❖ **Métodos:** Deben ser verbos o frases verbales en *camelCase* que indiquen claramente la acción (ej. calcularTotal(), obtenerId()).

- ❖ **Variables:** Nombres descriptivos en *camelCase* (ej. limiteMaximo, indiceActual). Se deben evitar letras sueltas

(como i, j), excepto en ciclos muy cortos. Las constantes deben ir en *UPPER_SNAKE_CASE* [3].

2) **Funciones:**

Deben ser pequeñas y hacer exactamente una sola cosa (Responsabilidad Única). Se recomienda que tengan el menor número de parámetros posibles (idealmente cero, uno o dos). Tres o más parámetros deben encapsularse en un objeto y deben evitar los "efectos secundarios", es decir, cambiar el estado de objetos ajenos a su propósito principal [1].

3) **Manejo de Excepciones:**

Es preferible lanzar excepciones en lugar de retornar códigos de error (ej. devolver -1 o null). Proporcionar contexto en las excepciones lanzadas con mensajes claros para facilitar el debugging, también nunca dejar un bloque catch vacío; al menos se debe registrar el error (logging) [2].

4) **Estructura del Código:**

- ❖ **Regla descendente (Stepdown Rule):** El código debe leerse como un artículo periodístico, de arriba hacia abajo. Las funciones más importantes o generales van arriba, y los detalles o funciones auxiliares debajo de ellas.
- ❖ Las variables deben declararse lo más cerca posible de su primer uso [1].

5) **Comentarios:**

El código debe explicarse a sí mismo. Un buen nombre de método o variable es mejor que un comentario. Solo se deben usar comentarios para explicar el "por qué" (decisiones de negocio, algoritmos matemáticos complejos o hacks necesarios), no el "qué" ni el "cómo".

Se debe evitar estrictamente dejar código comentado (código zombie) en producción [1].

6) **Formatos:** Mantener una indentación consistente (en Java generalmente se usan 4 espacios, no tabulaciones de tamaño variable). Agrupar líneas de código lógicamente relacionadas y separarlas mediante líneas en blanco para denotar diferentes conceptos visualmente.

Se recomienda seguir estándares de la industria para Java, como las convenciones originales de Sun Microsystems/Oracle o la Google Java Style Guide [3].

- Ejemplos de Código Mal Escrito vs Código Bien Escrito

Caso 1: Nomenclatura

```
// CÓDIGO MAL ESCRITO
public class GstAdp { no usages
    int d; // días en el refugio no usages
    boolean act; // estado activo no usages

    // Método para guardar
    public void gdrM(String n, String t) { no usages
        // lógica para guardar
    }
}
```

```
// CÓDIGO BIEN ESCRITO
public class GestorAdopciones { no usages
    int diasEnRefugio; no usages
    boolean estaDisponibleParaAdopcion; no usages

    public void guardarMascota(String nombreMascota, String tipoMascota) { no usages
        // lógica para guardar
    }
}
```

Caso 2: Funciones

```
// CÓDIGO MAL ESCRITO
public void procesarIngresoMascota(Mascota mascota, boolean tieneVacunas) {
    guardarEnBaseDeDatos(mascota);

    if (tieneVacunas) {
        registrarCarnetSanitario(mascota);
    } else {
        programarCitaVeterinario(mascota);
    }
}
```

```
// CÓDIGO BIEN ESCRITO
public void ingresarMascotaVacunada(Mascota mascota) {
    guardarEnBaseDeDatos(mascota);
    registrarCarnetSanitario(mascota);
}

public void ingresarMascotaSinVacunas(Mascota mascota) {
    guardarEnBaseDeDatos(mascota);
    programarCitaVeterinario(mascota);
}
```

Caso 3: Manejo de Excepciones

```
// CÓDIGO MAL ESCRITO
public String aprobarAdopcion(Usuario adoptante, Mascota mascota) { no usages
    if (mascota.estaEnTratamientoMedico()) {
        return "E_MASCOTA_ENFERMA"; // Devuelve un código de error como String
    }
    if (adoptante.tieneMultasPendientes()) {
        return "E_USUARIO_MULTADO";
    }
    // ... proceso de adopción ...
    return "E_OK";
}
```

```
// CÓDIGO BIEN ESCRITO
public void aprobarAdopcion(Usuario adoptante, Mascota mascota) throws AdopcionInvalidaException { no usages
    if (mascota.estaEnTratamientoMedico()) {
        throw new AdopcionInvalidaException("La mascota aún se está recuperando y no puede ser adoptada.");
    }

    if (adoptante.tieneMultasPendientes()) {
        throw new AdopcionInvalidaException("El usuario debe pagar sus multas antes de adoptar.");
    }

    // ... proceso de adopción normal y limpio ...
}
```

- **Ejemplo Práctico**

```
1 package edu.unl.cc.fundamentos.math;
2
3 public class Refactor { no usages
4
5     private static final int MINIMUM_VALID_VALUE = 1; 1 usage
6
7     public int sumPositiveIntegers(int firstOperand, int secondOperand) { no usages
8         validatePositive(firstOperand);
9         validatePositive(secondOperand);
10
11         return firstOperand + secondOperand;
12     }
13
14     private void validatePositive(int value) { 2 usages
15         if (value < MINIMUM_VALID_VALUE) {
16             throw new IllegalArgumentException("El valor debe ser positivo: " + value);
17         }
18     }
19 }
20
21 class ConsoleLogger { no usages
22     public void displayMessage(String message) { no usages
23         System.out.println("Mensaje: " + message);
24     }
25 }
```

- **Nombres significativos.**
Cambio: Se renombraron la clase (Utils → PositiveCalculator), el método (calc → sum) y los parámetros (a, b → firstOperand, secondOperand).
Razón: El código original era críptico. Los nuevos nombres revelan la intención y el propósito de cada elemento sin necesidad de leer la documentación.
- **Separación de responsabilidades (SRP).**
Cambio: Se extrajo la funcionalidad de imprimir mensajes a una clase independiente (ConsoleNotifier).
Razón: Una clase no debe calcular y mostrar datos al mismo tiempo. Al separarlas, si mañana decides cambiar la consola por una base de datos o un archivo, la lógica del cálculo no se verá afectada.
- **Manejo adecuado de errores (excepciones en lugar de mensajes por consola)**
Cambio: Se sustituyó el mensaje de error en consola por el lanzamiento de una excepción (IllegalArgumentException).
Razón: Imprimir un error y devolver 0 es una mala práctica porque el programa "miente" al usuario devolviendo un resultado que no es real. Las excepciones obligan al sistema a manejar el problema de forma segura.
- **Eliminar "magic numbers" si los hay.**
Cambio: Se creó la constante MINIMUM_ALLOWED_VALUE para reemplazar el uso directo del 0 en las comparaciones.
Razón: El 0 por sí solo no explica qué representa. La constante le otorga un nombre y un contexto, facilitando el mantenimiento si la regla de negocio cambia.
- **Simplificar la lógica (ej: evitar anidamientos innecesarios).**
Cambio: Se eliminó la estructura if-else anidada en favor de validaciones de salida rápida (Guard Clauses).

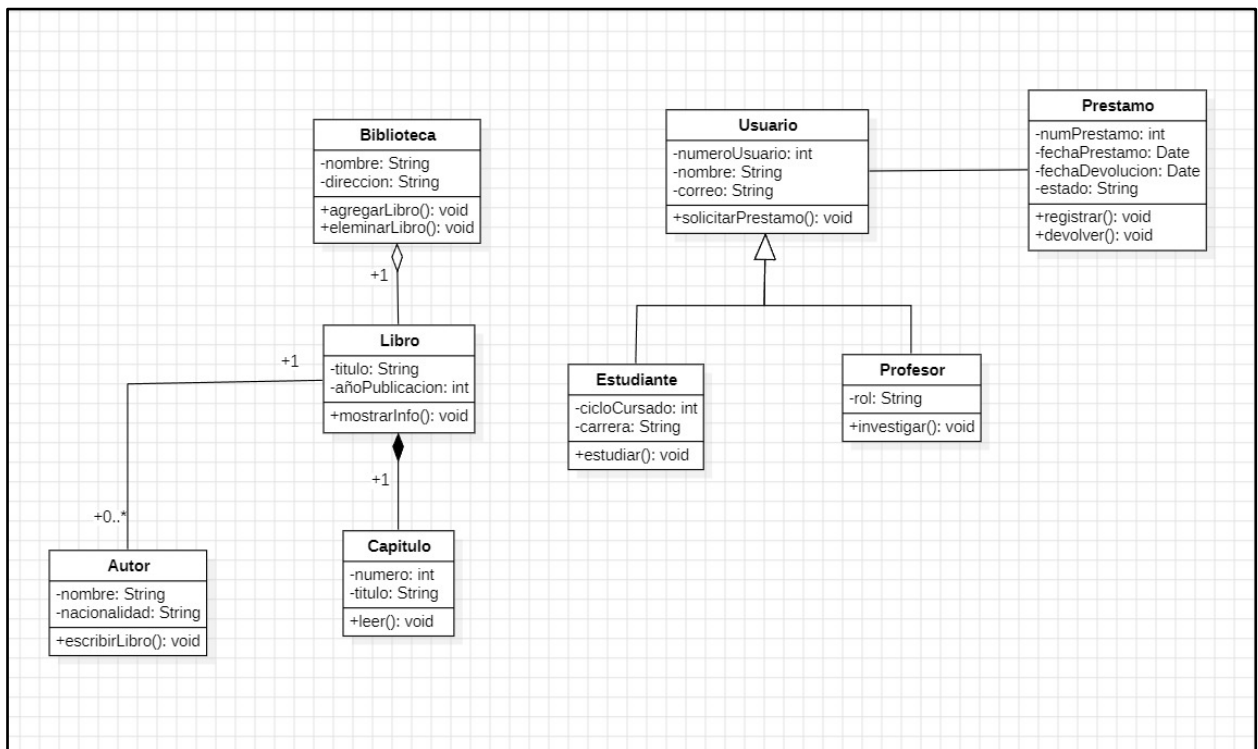
Razón: El anidamiento innecesario aumenta la complejidad visual. Al validar los errores al principio y "salir" del método si algo está mal, el flujo principal del código queda limpio y es mucho más fácil de leer.

2. Tipos de Relaciones en UML

- Define los 5 tipos de relaciones en UML (Asociación, Agregación, Composición, Generalización, Dependencia).
- Explica su significado, diferencias y cuándo aplicarlas (incluye ejemplos gráficos).

Tipo de Relación	Significado	Diferencias y cuándo aplicarla	Representación Gráfica
1. Asociación	Es una conexión donde dos clases independientes se comunican entre si	Diferencia: Ninguna clase es "dueña" de la otra. Sus ciclos de vida son independientes. Cuándo usarla: Cuando dos objetos necesitan hablarse, pero pueden existir por su cuenta (ej. Un usuario y un libro).	
2. Agregación	Es una relación "todo-parte" (un contenedor y sus elementos) pero de forma débil	Diferencia: La "parte" no depende del "todo" para vivir. Si el contenedor se destruye, las partes siguen existiendo. Cuándo usarla: Para agrupar elementos independientes (ej. Una biblioteca y sus libros).	
3. Composición	Es una relación "todo-parte" pero más fuerte y estricta	Diferencia: El ciclo de vida es atómico. La "parte" no puede existir sin el "todo". Si se destruye el principal, se destruyen sus componentes. Cuándo usarla: Cuando un elemento es vital para el otro (ej. Un libro y sus capítulos).	
4. Generalización	Es la relación de "es un...". Una clase hija hereda las características y comportamientos de una clase padre.	Diferencia: No relaciona objetos interactuando, sino que define jerarquías para evitar repetir código. Cuándo usarla: Para abstraer lógica común hacia una superclase (ej. Estudiante y Profesor heredan de Usuario).	
5. Dependencia	Indica que una clase (cliente) utiliza los servicios de otra (proveedor) de forma temporal.	Diferencia: Es el acoplamiento más débil. No hay propiedad ni conexión permanente. Cuándo usarla: Cuando se usa otra clase solo por un momento, por ejemplo, pasándola como parámetro en un método.	

- **Diseño Diagrama de Clase**



- **Conclusiones Personales**

- ✚ **Jean Carlos:**

En conclusión, aplicar Clean Code en este ejercicio va más allá de corregir la sintaxis de una clase Utils; es un verdadero cambio de mentalidad. Al transformar métodos genéricos como calc en nombres con intención y sustituir simples mensajes de consola por un manejo profesional de excepciones, asumimos la responsabilidad de programar con empatía para facilitar el trabajo futuro.

Conceptos como SOLID (al separar la lógica del cálculo de la impresión), DRY (al centralizar validaciones) y KISS (al simplificar la lógica eliminando anidamientos innecesarios) dejan de ser solo teoría para convertirse en herramientas indispensables que previenen errores y mejoran nuestro flujo de trabajo. En definitiva, esta práctica nos enseña que no basta con que el programa 'funcione' y sume dos números, sino que debemos estructurar un código limpio y profesional que hable por sí mismo.

- ✚ **Axel Jiménez:**

En conclusión, el desarrollo de software requiere la integración de la planificación teórica y la ejecución técnica. Los diagramas de clases en UML proporcionan una estructura lógica que define las relaciones entre los objetos de manera previa a la fase de programación. Durante la implementación en Java, la aplicación de principios de Clean Code (como SOLID, DRY y KISS) contribuye a la creación de un código legible y eficiente. Dominar ambas prácticas favorece la construcción de sistemas

escalables, disminuyendo la probabilidad de errores y optimizando su mantenimiento a largo plazo.

 Héctor Guerrero:

Para concluir, este trabajo nos deja la gran enseñanza de que escribir buen código no se trata de crear soluciones complejas, sino de hacer que lo difícil sea fácil de leer y entender. Aplicar reglas aparentemente sencillas, como nombrar bien las variables o crear funciones pequeñas, no es solo un requisito académico, sino la clave para construir software que pueda mantenerse en el tiempo sin convertirse en un dolor de cabeza para nosotros o para otros. En definitiva, adoptar el Clean Code nos prepara para dejar de programar solo para que un proyecto compile, y empezar a pensar como verdaderos profesionales de la computación que valoran la claridad y facilitan el trabajo en equipo.

 Irvin Armijos:

En conclusión, la Programación Limpia permite comprender que programar va más allá de hacer que un código funcione, enfocándose también en su claridad, organización y facilidad de mantenimiento a lo largo del tiempo.

La aplicación de principios como SOLID, DRY y KISS contribuye a reducir la complejidad, evitar errores y mejorar la calidad del software. Asimismo, un código bien estructurado facilita el trabajo en equipo y permite integrar o modificar funcionalidades de manera más eficiente.

 Erick Rogel:

En conclusión, aplicar Clean Code va más allá de seguir reglas de sintaxis en Java; es un verdadero cambio de mentalidad. Al comprender que el código se lee muchas más veces de las que se escribe, asumimos la responsabilidad de programar con empatía para facilitar el trabajo futuro. Conceptos como SOLID, DRY y KISS dejan de ser solo teoría de la clase de Programación Orientada a Objetos para convertirse en herramientas indispensables que previenen errores y mejoran nuestro trabajo. En definitiva, esta práctica nos enseña que no basta con que el programa "funcione", sino que debemos crear un código limpio y profesional que hable por sí mismo.

 Galo Benítez:

En conclusión, el diagrama UML ayuda a entender de una manera más fácil cómo funciona un sistema antes de ser desarrollado, como en este caso de la biblioteca, pues antes de tan siquiera empezar a programarlo. Gracias a este diseño, se puede ver cómo se relacionan las clases de los diferentes componentes, como lo son de las clases libros, autores, los usuarios y los préstamos dentro del sistema. También permite organizar mejor las ideas y evitar errores al momento de desarrollar el programa.

Además, permite identificar las relaciones como herencia, agregación y composición ayudan a representar situaciones de la vida real, por ejemplo, que un estudiante y un profesor son tipos de usuarios, o que un libro está formado por capítulos. En general, el UML hace que el sistema sea más claro, ordenado y sencillo de entender tanto para estudiantes como para programadores.

- **Declaración de Uso de IA**

Para este trabajo se utilizó la IA “Gemini, NotebookLM”, para mejorar la redacción del informe, mejorar la estructura, consultar dudas durante el desarrollo, y para la revisión de faltas ortográficas.

- **Referencias Bibliográficas**

[1] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ, USA: Prentice Hall, 2008.

[2] J. Bloch, *Effective Java*, 3rd ed. Boston, MA, USA: Addison-Wesley Professional, 2018.

[3] Oracle, "Code Conventions for the Java Programming Language," Oracle.com, 1999.

[En línea]. Disponible en:

<https://www.oracle.com/java/technologies/javase/codeconventions-introduction.html>.

[Accedido: 11-may-2026].